SaaS Landing Page

Product Requirements Document

Framer Motion Animations — Next.js (App Router)

Field	Value
Stack	Next.js 14+ App Router + TypeScript
Animation library	Framer Motion v11
Animation scope	Hero fade + slide, Scroll-triggered reveals
Target	AI-assisted implementation (Claude Sonnet 4.6)
Document version	1.0

1. Document Purpose

This PRD is a self-contained specification designed to be fed directly to Claude Sonnet 4.6 inside VS Code. It contains every instruction, code pattern, file structure, and acceptance criterion required to build a production-ready animated SaaS landing page end-to-end — without back-and-forth clarification.

> **How to use:** Paste the entire contents of this document into Claude Sonnet 4.6 as the initial prompt. Claude will scaffold the project, create all files, and implement every section described below.

---

2. Tech Stack & Prerequisites

2.1 Required versions

Package	Version / Note
Node.js	>= 18.17
Next.js	14+ (App Router)
TypeScript	5+
framer-motion	v11 (npm install framer-motion)
Tailwind CSS	v3 (via create-next-app)

2.2 Project bootstrap command

Run the following command to create the project before invoking Claude:

```
npx create-next-app@latest saas-landing \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir \
  --import-alias '@/*'

cd saas-landing
npm install framer-motion
```

---

3. File & Folder Structure

Claude must create the project with exactly this structure:

```
src/
  app/
    layout.tsx          # Root layout — LazyMotion wrapper lives here
    page.tsx            # Assembles all landing page sections
    globals.css         # Tailwind base + custom CSS variables
  components/
    layout/
      Navbar.tsx        # Sticky top navigation
    sections/
      HeroSection.tsx   # Hero with fade+slide animation
      FeaturesSection.tsx  # Feature cards with scroll reveal
      SocialProofSection.tsx  # Logos / testimonials scroll reveal
      PricingSection.tsx   # Pricing cards scroll reveal
      CTASection.tsx       # Final CTA scroll reveal
      FooterSection.tsx    # Footer
    ui/
      AnimatedSection.tsx  # Reusable scroll-reveal wrapper
      AnimatedText.tsx     # Reusable animated headline wrapper
  lib/
    animations.ts       # All animation variants (single source of truth)
```

---

4. Animation Specification

4.1 lib/animations.ts — variants library

All animation variants MUST be defined in this single file and imported everywhere. No inline animation objects in component files.

```
// src/lib/animations.ts

export const fadeUp = {
  initial: { opacity: 0, y: 30 },
  animate: { opacity: 1, y: 0 },
};

export const fadeIn = {
  initial: { opacity: 0 },
  animate: { opacity: 1 },
};

export const slideInLeft = {
  initial: { opacity: 0, x: -40 },
  animate: { opacity: 1, x: 0 },
};
```

```
export const slideInRight = {
  initial: { opacity: 0, x: 40 },
  animate: { opacity: 1, x: 0 },
};

// Scroll-reveal — use with whileInView
export const scrollReveal = {
  hidden: { opacity: 0, y: 40 },
  visible: { opacity: 1, y: 0 },
};

// Stagger container — wraps a list of child items
export const staggerContainer = {
  hidden: {},
  visible: {
    transition: {
      staggerChildren: 0.1,
      delayChildren: 0.15,
    },
  },
};

// Stagger child item — must be used with staggerContainer
export const staggerItem = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.45 } },
};

// Timing presets
export const transitions = {
  hero:   { duration: 0.65, ease: 'easeOut' },
  reveal: { duration: 0.55, ease: 'easeOut' },
  fast:   { duration: 0.2,  ease: 'easeOut' },
  spring: { type: 'spring', stiffness: 300, damping: 22 },
};
```

4.2 Timing reference

Element	Duration / Delay
Hero headline	0.65s — delay 0s
Hero subtext	0.6s — delay 0.15s
Hero CTA button	0.5s — delay 0.3s
Scroll section reveal	0.55s — trigger at 25% visible
Stagger children	100ms between each child
Button hover scale	0.15s spring (whileHover: scale 1.05)
Button tap scale	0.1s spring (whileTap: scale 0.97)

5. Root Layout — app/layout.tsx

The LazyMotion wrapper must live at the root layout so all child pages inherit it. Use domAnimation (not domMax) to keep bundle size optimal for a landing page.

```
// src/app/layout.tsx
import type { Metadata } from 'next';
import { LazyMotion, domAnimation } from 'framer-motion';
import { Inter } from 'next/font/google';
import './globals.css';

const inter = Inter({ subsets: ['latin'] });

export const metadata: Metadata = {
  title: 'Your SaaS Product',
  description: 'Short product description here',
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang='en'>
      <body className={inter.className}>
        <LazyMotion features={domAnimation}>
          {children}
        </LazyMotion>
      </body>
    </html>
  );
}
```

Why LazyMotion? Using LazyMotion + domAnimation cuts the Framer Motion bundle from ~50KB to ~25KB gzipped. Use m.div instead of motion.div inside LazyMotion boundaries.

6. Reusable Animation Components

6.1 AnimatedSection — scroll-reveal wrapper

Every landing page section below the hero should be wrapped in this component. It uses whileInView to trigger once when the section enters the viewport.

```
// src/components/ui/AnimatedSection.tsx
'use client';

import { m } from 'framer-motion';
import { scrollReveal, transitions } from '@/lib/animations';

interface AnimatedSectionProps {
  children: React.ReactNode;
  className?: string;
  delay?: number;
}

export default function AnimatedSection({
  children,
  className,
  delay = 0,
}: AnimatedSectionProps) {
  return (
    <m.section
      variants={scrollReveal}
      initial='hidden'
      whileInView='visible'
      viewport={{ once: true, amount: 0.25 }}
      transition={{ ...transitions.reveal, delay }}
      className={className}
    >
      {children}
    </m.section>
  );
}
```

6.2 AnimatedText — headline wrapper

Use this for any headline or subtext that should animate in. Accepts a 'as' prop to control the HTML tag (h1, h2, p, span).

```
// src/components/ui/AnimatedText.tsx
'use client';

import { m } from 'framer-motion';
import { fadeUp, transitions } from '@/lib/animations';

interface AnimatedTextProps {
  children: React.ReactNode;
  as?: 'h1' | 'h2' | 'h3' | 'p' | 'span';
  delay?: number;
  className?: string;
}

export default function AnimatedText({
```

```
  children,
  as: Tag = 'p',
  delay = 0,
  className,
}: AnimatedTextProps) {
  const MotionTag = m[Tag];
  return (
    <MotionTag
      {...fadeUp}
      transition={{ ...transitions.hero, delay }}
      className={className}
    >
      {children}
    </MotionTag>
  );
}
```

---

7. Section Specs

7.1 HeroSection

The hero is the most important section. It animates on initial page load (not scroll-triggered). The headline, subtext, and CTA button stagger in sequence.

> **Rule:** Hero uses animate (not whileInView) because it must play immediately on load, not when scrolled to.

```
// src/components/sections/HeroSection.tsx
'use client';

import { m } from 'framer-motion';
import { fadeUp, transitions } from '@/lib/animations';

export default function HeroSection() {
  return (
    <section className='min-h-screen flex flex-col items-center justify-center
                        text-center px-6 py-24 bg-gradient-to-br
                        from-slate-900 via-slate-800 to-slate-900'>

      {/* Badge */}
      <m.div
        {...fadeUp}
        transition={{ ...transitions.hero, delay: 0 }}
        className='mb-4 px-4 py-1 rounded-full border border-slate-600
                   text-slate-300 text-sm'
      >
        Now in public beta
```

```
      </m.div>

      {/* Headline */}
      <m.h1
        {...fadeUp}
        transition={{ ...transitions.hero, delay: 0.1 }}
        className='text-5xl md:text-7xl font-bold text-white max-w-4xl
                   leading-tight mb-6'
      >
        Your product headline
        <span className='text-blue-400'> goes here</span>
      </m.h1>

      {/* Subtext */}
      <m.p
        {...fadeUp}
        transition={{ ...transitions.hero, delay: 0.25 }}
        className='text-xl text-slate-300 max-w-2xl mb-10'
      >
        Supporting description that explains the value proposition
        clearly in one or two sentences.
      </m.p>

      {/* CTA Buttons */}
      <m.div
        {...fadeUp}
        transition={{ ...transitions.hero, delay: 0.4 }}
        className='flex flex-col sm:flex-row gap-4'
      >
        <m.button
          whileHover={{ scale: 1.05 }}
          whileTap={{ scale: 0.97 }}
          transition={transitions.spring}
          className='px-8 py-3 bg-blue-500 hover:bg-blue-600 text-white
                     rounded-lg font-semibold text-lg'
        >
          Get started free
        </m.button>
        <m.button
          whileHover={{ scale: 1.03 }}
          whileTap={{ scale: 0.97 }}
          transition={transitions.spring}
          className='px-8 py-3 border border-slate-500 text-slate-200
                     rounded-lg font-semibold text-lg hover:border-slate-400'
        >
          See a demo
        </m.button>
      </m.div>
    </section>
  );
}
```

7.2 FeaturesSection

Six feature cards in a 3-column grid. The section heading uses AnimatedSection. The card grid uses stagger variants so cards cascade in one by one.

```
// src/components/sections/FeaturesSection.tsx
'use client';

import { m } from 'framer-motion';
import AnimatedSection from '@/components/ui/AnimatedSection';
import { staggerContainer, staggerItem } from '@/lib/animations';

const features = [
  { icon: '⚡', title: 'Feature One', desc: 'Short description of this feature
benefit.' },
  { icon: '□', title: 'Feature Two', desc: 'Short description of this feature
benefit.' },
  { icon: '□', title: 'Feature Three', desc: 'Short description of this feature
benefit.' },
  { icon: '□', title: 'Feature Four', desc: 'Short description of this feature
benefit.' },
  { icon: '□', title: 'Feature Five', desc: 'Short description of this feature
benefit.' },
  { icon: '□', title: 'Feature Six', desc: 'Short description of this feature
benefit.' },
];

export default function FeaturesSection() {
  return (
    <AnimatedSection className='py-24 px-6 bg-white'>
      <div className='max-w-6xl mx-auto'>
        <h2 className='text-4xl font-bold text-center mb-4 text-slate-900'>
          Everything you need
        </h2>
        <p className='text-center text-slate-500 mb-16 text-lg'>
          Supporting subheading for the features section.
        </p>

        <m.ul
          variants={staggerContainer}
          initial='hidden'
          whileInView='visible'
          viewport={{ once: true, amount: 0.15 }}
          className='grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8
                     list-none m-0 p-0'
        >
          {features.map((f) => (
            <m.li
              key={f.title}
              variants={staggerItem}
              className='p-6 rounded-2xl border border-slate-100
                         bg-slate-50 hover:shadow-md transition-shadow'
            >
              <span className='text-3xl mb-4 block'>{f.icon}</span>
              <h3 className='font-semibold text-lg mb-2 text-slate-800'>
                {f.title}
              </h3>
```

```
              <p className='text-slate-500 text-sm leading-relaxed'>{f.desc}</p>
            </m.li>
          ))}
        </m.ul>
      </div>
    </AnimatedSection>
  );
}
```

7.3 SocialProofSection

Customer logos row + one or two testimonial quotes. Entire section wrapped in AnimatedSection for a simple reveal.

```
// src/components/sections/SocialProofSection.tsx
'use client';

import AnimatedSection from '@/components/ui/AnimatedSection';

const logos = ['Acme Co', 'Globex', 'Initech', 'Umbrella', 'Hooli', 'Pied Piper'];

export default function SocialProofSection() {
  return (
    <AnimatedSection className='py-16 px-6 bg-slate-50'>
      <div className='max-w-5xl mx-auto text-center'>
        <p className='text-slate-400 text-sm uppercase tracking-widest mb-8'>
          Trusted by teams at
        </p>
        <div className='flex flex-wrap justify-center gap-10 mb-16'>
          {logos.map((logo) => (
            <span key={logo}
              className='text-slate-300 font-bold text-xl tracking-tight'>
              {logo}
            </span>
          ))}
        </div>
        <blockquote className='text-xl text-slate-600 italic max-w-2xl mx-auto'>
          &ldquo;This product completely changed how our team works.
          We shipped 3x faster in the first month.&rdquo;
        </blockquote>
        <p className='mt-4 text-slate-400 text-sm'>
          — Jane Smith, CTO at Acme Co
        </p>
      </div>
    </AnimatedSection>
  );
}
```

7.4 PricingSection

Three pricing tiers (Free, Pro, Enterprise). Each card uses stagger. Highlight the middle card visually with a border accent.

```
// src/components/sections/PricingSection.tsx
'use client';

import { m } from 'framer-motion';
import AnimatedSection from '@/components/ui/AnimatedSection';
import { staggerContainer, staggerItem } from '@/lib/animations';

const plans = [
  { name: 'Free', price: '$0', features: ['Up to 3 projects', '1 GB storage',
'Community support'], cta: 'Get started', accent: false },
  { name: 'Pro', price: '$29', features: ['Unlimited projects', '50 GB storage',
'Priority support', 'Advanced analytics'], cta: 'Start free trial', accent: true },
  { name: 'Enterprise', price: 'Custom', features: ['Everything in Pro', 'SSO /
SAML', 'SLA guarantee', 'Dedicated CSM'], cta: 'Contact sales', accent: false },
];

export default function PricingSection() {
  return (
    <AnimatedSection className='py-24 px-6 bg-white'>
      <div className='max-w-5xl mx-auto'>
        <h2 className='text-4xl font-bold text-center mb-4 text-slate-900'>
          Simple, transparent pricing
        </h2>
        <p className='text-center text-slate-500 mb-16'>
          No hidden fees. Cancel anytime.
        </p>
        <m.div
          variants={staggerContainer}
          initial='hidden'
          whileInView='visible'
          viewport={{ once: true, amount: 0.2 }}
          className='grid grid-cols-1 md:grid-cols-3 gap-8'
        >
          {plans.map((plan) => (
            <m.div
              key={plan.name}
              variants={staggerItem}
              className={`p-8 rounded-2xl border-2 ${
                plan.accent
                  ? 'border-blue-500 shadow-lg shadow-blue-100'
                  : 'border-slate-100'
              }`}
            >
              {plan.accent && (
                <span className='text-xs font-semibold text-blue-500
                                 uppercase tracking-wider'>
                  Most popular
                </span>
              )}
              <h3 className='text-xl font-bold mt-2 mb-1'>{plan.name}</h3>
              <p className='text-4xl font-bold mb-6'>{plan.price}
                <span className='text-base font-normal text-slate-400'>
```

```
                {plan.price !== 'Custom' ? '/mo' : ''}
              </span>
            </p>
            <ul className='space-y-2 mb-8'>
              {plan.features.map((f) => (
                <li key={f} className='text-slate-600 text-sm flex gap-2'>
                  <span className='text-green-500'>✓</span> {f}
                </li>
              ))}
            </ul>
            <button className={`w-full py-3 rounded-lg font-semibold ${
              plan.accent
                ? 'bg-blue-500 text-white hover:bg-blue-600'
                : 'border border-slate-200 text-slate-700 hover:bg-slate-50'
            }`}>
              {plan.cta}
            </button>
          </m.div>
        ))}
      </m.div>
    </div>
  </AnimatedSection>
);
}
```

7.5 CTASection

A final call-to-action banner before the footer. Simple centered layout, wrapped in AnimatedSection.

```
// src/components/sections/CTASection.tsx
'use client';

import { m } from 'framer-motion';
import AnimatedSection from '@/components/ui/AnimatedSection';
import { transitions } from '@/lib/animations';

export default function CTASection() {
  return (
    <AnimatedSection className='py-24 px-6 bg-slate-900 text-center'>
      <h2 className='text-4xl font-bold text-white mb-4'>
        Ready to get started?
      </h2>
      <p className='text-slate-300 mb-10 text-lg max-w-xl mx-auto'>
        Join thousands of teams already using this product.
      </p>
      <m.button
        whileHover={{ scale: 1.05 }}
        whileTap={{ scale: 0.97 }}
        transition={transitions.spring}
        className='px-10 py-4 bg-blue-500 hover:bg-blue-600 text-white
                   rounded-lg font-semibold text-lg'
```

```
      >
        Start for free
      </m.button>
    </AnimatedSection>
  );
}
```

---

8. Page Assembly — app/page.tsx

The root page.tsx assembles all sections in order. It is a Server Component (no 'use client' directive).

```
// src/app/page.tsx
import Navbar from '@/components/layout/Navbar';
import HeroSection from '@/components/sections/HeroSection';
import FeaturesSection from '@/components/sections/FeaturesSection';
import SocialProofSection from '@/components/sections/SocialProofSection';
import PricingSection from '@/components/sections/PricingSection';
import CTASection from '@/components/sections/CTASection';
import FooterSection from '@/components/sections/FooterSection';

export default function LandingPage() {
  return (
    <main>
      <Navbar />
      <HeroSection />
      <FeaturesSection />
      <SocialProofSection />
      <PricingSection />
      <CTASection />
      <FooterSection />
    </main>
  );
}
```

---

9. Implementation Rules for Claude

9.1 Animation rules

- Always import m from 'framer-motion', not motion, inside LazyMotion boundaries
- Never write inline animation objects — always import from lib/animations.ts
- Hero animations use initial + animate (page-load driven)
- All sections below the hero use whileInView with viewport={{ once: true, amount: 0.25 }}
- Every whileInView must have once: true — sections must not re-animate on scroll-up

- Stagger wrappers (m.ul, m.div with staggerContainer) must also be whileInView, not animate
- Button hover: whileHover={{ scale: 1.05 }}, tap: whileTap={{ scale: 0.97 }}

9.2 Performance rules

- Only animate transform (x, y, scale, rotate) and opacity — never width, height, top, left
- Keep hero total animation duration under 0.7s (last element finishes animating)
- Use LazyMotion + domAnimation in root layout — never import from 'framer-motion/client'

9.3 Accessibility rules

- Add useReducedMotion check in AnimatedSection and AnimatedText
- When shouldReduceMotion is true, set duration to 0.01 and y offset to 0

```
// Pattern for reduced motion in AnimatedSection.tsx
import { useReducedMotion } from 'framer-motion';

const shouldReduce = useReducedMotion();

// Then override transition:
transition={{
  duration: shouldReduce ? 0.01 : transitions.reveal.duration,
  ease: transitions.reveal.ease,
  delay: shouldReduce ? 0 : delay,
}}

// And override initial:
initial={{ opacity: 0, y: shouldReduce ? 0 : 40 }}
```

9.4 Next.js App Router rules

- page.tsx is a Server Component — no 'use client'
- All components using Framer Motion must have 'use client' at the top
- layout.tsx wraps children with LazyMotion — this is a Client Component
- Do not use useEffect or useState in Server Components

---

10. Acceptance Criteria

The implementation is complete when ALL of the following are true:

1. npm run dev starts without TypeScript or lint errors
2. Hero headline, subtext, and CTA animate in on initial page load
3. All 3 hero elements stagger with the specified delays
4. FeaturesSection cards cascade in one-by-one with 100ms stagger when scrolled into view
5. SocialProofSection, PricingSection, and CTASection each reveal on scroll
6. All scroll-triggered animations fire only once (not on scroll-back-up)
7. Buttons have hover scale 1.05 and tap scale 0.97
8. With 'Reduce Motion' enabled in OS, no translate/scale animations play (only opacity)
9. No animation variant objects are defined inline in component files
10. npm run build completes with no errors

---

11. Claude Prompt Template

Copy and paste the following as your first message to Claude Sonnet 4.6 in VS Code:

```
You are building a SaaS landing page using Next.js 14 App Router + TypeScript +
Tailwind CSS + Framer Motion v11.

Follow this PRD exactly. Create ALL files listed in Section 3. Use the animation
variants from Section 4 — never define animation objects inline. Implement all
sections from Section 7. Follow all rules in Section 9. The implementation is done
when all acceptance criteria in Section 10 pass.

Start by creating src/lib/animations.ts, then src/app/layout.tsx, then all
components in order. Show me each file as you create it.
```

Appendix: Quick Reference

Framer Motion imports cheat sheet

```
// Inside LazyMotion boundaries — use m.* instead of motion.*
import { m, AnimatePresence, useReducedMotion,
         useScroll, useTransform } from 'framer-motion';

// Root layout only
import { LazyMotion, domAnimation } from 'framer-motion';

// Usage
<m.div initial={{ opacity: 0 }} animate={{ opacity: 1 }}>
  // NOT <motion.div> inside LazyMotion
```

```
</m.div>
```

Common mistakes to avoid

- Using motion.div instead of m.div inside LazyMotion — causes bundle duplication
- Forgetting 'use client' on any component that imports from framer-motion
- Setting once: false on viewport — sections re-animate on every scroll
- Animating height or padding — causes layout thrash, use scaleY instead
- Defining variants inside component render — recreated on every render, hurts performance

— End of Document —